Detailed & Production-Grade Rewritten Specification

Based on the uploaded hiring challenge document and expanded using modern multi-agent orchestration patterns, runtime architectures, and production agent platform practices from LangGraph, CrewAI, OpenClaw, and AutoGen ecosystems. (Ulysses32)

AI Engineer Hiring Challenge — Expanded Technical Specification

Project Title

AI Agent Orchestration Platform

---

1. Objective

Build a fully functional multi-agent orchestration platform where users can:

- Create and configure AI agents
- Define how agents behave
- Connect agents together into workflows
- Allow agents to collaborate autonomously
- Enable asynchronous communication between agents
- Integrate real external tools
- Persist memory and execution history
- Interact with at least one agent through an external messaging platform
- Monitor workflows and execution in real time through a web UI

This is NOT a mockup challenge.

The system must execute real runtime logic using an actual orchestration framework such as:

- LangGraph
- CrewAI
- AutoGen
- OpenClaw
- OR a custom runtime implementation

The platform must demonstrate real autonomous multi-agent execution.

---

2. Core Deliverables

The repository submission MUST include:

2.1 Source Repository

A complete working repository containing:

- Backend
- Frontend
- Runtime engine
- Database integration
- Messaging integration
- Workflow orchestration
- Tool execution system
- Logging and monitoring
- Documentation

2.2 README

The README must include:

Mandatory Sections

Architecture Overview

- High-level architecture diagram
- Component explanation
- Runtime explanation
- Message flow explanation

- Agent lifecycle explanation

Technology Choices

Explain:

- Why the chosen framework was selected
- Why the frontend stack was selected
- Why the backend stack was selected
- Why the database was selected

Setup Instructions

Provide:

- One-command setup
- Environment configuration
- API key configuration
- Messaging channel setup
- Database initialization

Execution Instructions

Explain:

- How to start frontend
- How to start backend
- How to create agents
- How to execute workflows
- How to test messaging integration

Extensibility Guide

Explain:

- How to add tools
- How to add workflow templates
- How to add new messaging channels
- How to add new agent types

2.3 Recorded Demo

A video or GIF demonstrating:

Mandatory Demo Flow

1. Create agents
2. Configure agents
3. Connect agents into workflow
4. Trigger workflow
5. Show inter-agent communication
6. Show tool execution
7. Show memory persistence
8. Show logs/monitoring
9. Interact with an agent via external messaging channel
10. Demonstrate successful task completion

---

3. Functional Requirements

3.1 Agent Management

The platform MUST support complete CRUD operations.

Agent Creation Fields

Each agent must support:

Field	Required	Description
Agent Name	Yes	Human-readable unique name
Description	Yes	Purpose of the agent
Role	Yes	Agent responsibility
System Prompt	Yes	Core behavior instructions
Model Provider	Yes	OpenAI/Anthropic/Ollama/etc
Model Name	Yes	GPT-4o / Claude / etc

Temperature	Optional	Creativity level
Max Tokens	Optional	Response limit
Tools	Optional	Attached executable tools
Memory Enabled	Yes	Persistent memory toggle
Schedule	Optional	Cron or interval execution
Channels	Optional	Slack/Telegram/etc
Guardrails	Optional	Behavioral restrictions
Rate Limits	Optional	Execution limits
Active Status	Yes	Enabled/Disabled

3.2 Agent Runtime

The runtime MUST:

- Execute real LLM calls
- Execute tools
- Maintain agent state
- Support asynchronous execution
- Support message passing
- Support retries
- Support failure handling
- Support memory retrieval
- Support workflow continuation

Mock implementations are NOT acceptable.

3.3 Multi-Agent Collaboration

Agents must:

- Send messages to each other
- Share context

- Delegate subtasks
- Continue execution asynchronously
- Support feedback loops
- Support conditional branching

Minimum requirement:

- 2 collaborating agents

Recommended:

- 3+ specialized agents

3.4 Workflow Builder

A visual workflow builder is REQUIRED.

Workflow Features

The workflow editor must support:

Feature	Required
Drag-and-drop nodes	Yes
Agent nodes	Yes
Conditional routing	Yes
Parallel branches	Yes
Retry loops	Yes
Human approval step	Optional
Trigger nodes	Yes
Tool execution nodes	Yes

Start/End nodes	Yes

Workflow Execution Requirements

The workflow engine must support:

- Stateful execution
- Intermediate persistence
- Error recovery
- Step-level logging
- Real-time updates

3.5 Messaging Integration

At least ONE external messaging integration is REQUIRED.

Allowed channels:

- Telegram
- Slack
- WhatsApp

Telegram is strongly recommended due to easiest setup.

Messaging Requirements

The integration must support:

Feature	Required
Receive messages	Yes
Send responses	Yes
Maintain conversation history	Yes
Route messages to runtime	Yes

Persist messages	Yes
Real-time response	Yes

3.6 Tool Execution

Agents must execute REAL tools.

Examples:

- Web search
- File operations
- Calculator
- Weather API
- Database queries
- HTTP requests
- Browser automation

At least 3 real tools are required.

3.7 Memory System

Agents must support persistent memory.

Memory may include:

- Conversation history
- Task history
- Shared workflow state
- Agent-specific notes

Acceptable implementations:

- PostgreSQL
- Redis
- ChromaDB
- SQLite
- Vector DB

3.8 Monitoring & Observability

The platform must include live monitoring.

Required Monitoring Features

Feature	Required
Execution logs	Yes
Inter-agent messages	Yes
Workflow status	Yes
Token usage	Yes
Cost tracking	Yes
Agent states	Yes
Failure logs	Yes
Tool execution logs	Yes

---

4. UI/UX Requirements

The system MUST include a modern web UI.

Required Screens

Dashboard

Display:

- Active agents
- Running workflows
- Token usage
- System status
- Recent executions

Agent Management Screen

Support:

- Create/Edit/Delete agents
- Attach tools
- Configure prompts
- Enable/disable memory
- Configure channels

Workflow Builder Screen

Support:

- Visual graph editor
- Node connections
- Conditional logic
- Execution visualization

Monitoring Screen

Support:

- Live logs
- Runtime events
- Message tracing
- Tool execution history

Chat Interface

Support:

- Real-time chat with agent
- Messaging history
- External channel sync

---

5. Recommended Architecture

5.1 Preferred Stack

This is NOT mandatory, but strongly recommended for implementation speed and maintainability.

Frontend

Recommended:

- Next.js
- React
- TypeScript
- TailwindCSS
- React Flow (workflow builder)

Backend

Recommended:

- FastAPI
- Python
- AsyncIO

Reason:
Most orchestration frameworks are Python-native.

Runtime Layer

Recommended:

- LangGraph
  OR
- CrewAI

LangGraph is better for:

- Stateful workflows
- Complex graph orchestration
- Retry systems
- Conditional routing

CrewAI is better for:

- Fast setup
- Role-based collaboration
- Simpler orchestration

(openagents.org)

Database

Recommended:

- PostgreSQL

Optional:

- Redis for queues/caching

Realtime Layer

Recommended:

- WebSockets
  OR
- Socket.IO

Messaging Integration

Recommended:

- Telegram Bot API

Reason:
Simpler setup than WhatsApp.

---

6. Minimum Required Workflows

The system MUST include at least TWO prebuilt workflow templates.

Template 1 — Research Workflow

Agents

- Research Agent
- Summarizer Agent
- Reviewer Agent

Flow

1. User submits topic
2. Research agent gathers information
3. Summarizer creates report
4. Reviewer validates output
5. Final result returned

Template 2 — Customer Support Workflow

Agents

- Intake Agent
- Knowledge Agent
- Escalation Agent

Flow

1. User sends issue via Telegram
2. Intake agent classifies issue
3. Knowledge agent responds
4. Escalation agent handles unresolved cases

---

7. Non-Functional Requirements

7.1 Code Quality

Code must include:

- Modular architecture
- Separation of concerns
- Dependency injection
- Typed APIs
- Environment-based configuration
- Error handling
- Structured logging

7.2 Testing

Minimum required tests:

Test Type	Required
Agent creation	Yes
Workflow execution	Yes
Message delivery	Yes
Tool execution	Yes
API routes	Yes

7.3 Local Execution

The project MUST run locally.

Preferred startup:

```
docker compose up
```

OR

```
make dev
```

OR

```
./start.sh
```

Single-command startup is strongly preferred.

---

8. Suggested Repository Structure

```
project-root/
│
├── frontend/
├── backend/
├── runtime/
├── workflows/
```

```
├── agents/
├── tools/
├── messaging/
├── database/
├── tests/
├── docker/
├── docs/
│
├── docker-compose.yml
├── README.md
├── .env.example
└── Makefile
```

---

9. Suggested API Design

Agent APIs

```
POST   /agents
GET    /agents
GET    /agents/:id
PUT    /agents/:id
DELETE /agents/:id
```

Workflow APIs

```
POST /workflows
POST /workflows/:id/execute
GET  /workflows/:id/status
```

Messaging APIs

```
POST /webhooks/telegram
POST /webhooks/slack
```

Monitoring APIs

GET /logs
GET /metrics
GET /executions

---

10. Recommended Internal Architecture

Layered Design

Frontend UI
↓
API Layer
↓
Orchestration Layer
↓
Agent Runtime
↓
Tools + Memory + Messaging
↓
Persistence Layer

---

11. Realistic Scope Definition

To avoid overengineering, candidates SHOULD prioritize:

High Priority

- End-to-end execution
- Stable runtime
- Agent collaboration

- Messaging integration
- Monitoring
- Workflow execution

Medium Priority

- Beautiful UI
- Advanced memory
- Advanced analytics

Low Priority

- Enterprise auth
- Kubernetes
- Multi-tenant systems
- Production-scale optimization

---

12. Explicit Acceptance Criteria

The submission will be considered COMPLETE only if:

Requirement	Mandatory
Real agent runtime	Yes
Real LLM execution	Yes
Multi-agent collaboration	Yes
External messaging integration	Yes
Workflow builder	Yes
Persistent memory	Yes
Monitoring dashboard	Yes
End-to-end demo	Yes
Local execution	Yes

README	Yes
Tests	Yes

---

13. Recommended Technical Decisions (Strongly Suggested)

Best Overall Stack for This Challenge

Recommended Architecture

Layer	Recommendation
Frontend	Next.js + React Flow
Backend	FastAPI
Runtime	LangGraph
Queue	Celery or AsyncIO
Database	PostgreSQL
Cache	Redis
Messaging	Telegram
Realtime	WebSockets
ORM	SQLAlchemy
Auth	Optional JWT

Why LangGraph is Likely the Best Choice

LangGraph is particularly suitable because the challenge explicitly requires:

- Stateful workflows
- Conditional execution
- Feedback loops
- Persistent execution
- Multi-agent orchestration
- Async graph execution

These align directly with LangGraph's strengths. (openagents.org)

---

14. Hidden Evaluation Factors (Important)

The original challenge document does NOT explicitly say these, but interviewers will almost certainly evaluate them:

14.1 Runtime Reliability

Does the system fail gracefully?

14.2 Architecture Maturity

Is the system modular and scalable?

14.3 Agent Isolation

Can agents operate independently?

14.4 Observability

Can developers debug workflows easily?

14.5 Extensibility

Can new tools/channels/workflows be added quickly?

---

15. Common Failure Cases to Avoid

Major Red Flags

Avoid:

- Hardcoded workflows
- Fake agent communication
- UI-only implementation
- Synchronous blocking execution
- No persistence
- No monitoring
- No retry handling
- No async orchestration
- No external messaging integration
- Massive monolithic backend

---

16. Final Recommended MVP Scope

If time-constrained, the BEST submission would include:

Core Stack

- FastAPI
- LangGraph
- PostgreSQL
- Next.js
- Telegram Bot
- React Flow

Features

- 3 agents
- 2 workflow templates
- Real async execution
- Persistent memory
- Live logs
- Telegram interaction
- Tool execution
- Workflow visualization

That is more than sufficient for a very strong submission.

---

17. Final Validation Checklist (Foolproof Build Checklist)

Runtime

- Real orchestration framework used
- Async execution works
- Agents communicate
- Tool execution works

UI

- Agent CRUD works
- Workflow builder works
- Monitoring dashboard works

Messaging

- Telegram/Slack/WhatsApp connected
- Messages persisted
- Conversations visible in UI

Persistence

- Agents persisted
- Workflows persisted
- Messages persisted
- Logs persisted

Monitoring

- Live logs visible
- Token tracking visible
- Workflow states visible

Documentation

- README complete
- Setup instructions verified
- Demo recorded

Testing

- Critical APIs tested
- Workflow execution tested
- Messaging integration tested

Additional Architecture References

- LangGraph
- CrewAI
- AutoGen Studio Research Paper
- OpenClaw Framework Overview
- React Flow
- FastAPI
- Telegram Bot API

Supporting references used for architectural expansion and validation: (Ulysses32)